

Datasheet for Black Box Optimization (BBO) Capstone Project

Dataset Name: Black Box Optimization Benchmark Functions with Bayesian Optimization

Version: 1.0

Date Created: February 2026

Last Updated: April 2026

Authors: Kalpesh Jadhav

Contact: kalpeshjadhav.jde@gmail.com

GitHub Repository:

1. MOTIVATION

1.1 Purpose and Task

This dataset was created to support **Black Box Optimization (BBO)** research, specifically to develop and evaluate **Bayesian Optimization strategies** for finding optimal input points without explicit knowledge of objective function structure or gradients.

1.2 Primary Research Questions

The dataset enables investigation of:

- How effectively can Gaussian Process (GP) surrogate models guide optimization across varying dimensionalities (2D to 8D)?
- Does the Upper Confidence Bound (UCB) acquisition function consistently find good solutions?
- How does optimization performance scale with problem dimensionality?
- Can iterative refinement with limited evaluations (1 per round) find near-optimal solutions?

1.3 Application Domain

This is a **synthetic benchmark optimization** problem, suitable for:

- Testing and validating optimization algorithms
- Educational demonstrations of Bayesian Optimization
- Benchmarking surrogate model quality
- Academic research in hyperparameter optimization and black-box optimization

1.4 Related Work

This project adapts established BBO benchmark functions from optimization literature to create a comprehensive test suite for evaluating Bayesian Optimization strategies in varying dimensionality.

2. COMPOSITION

2.1 Dataset Structure

The dataset consists of:

Component	Description
-----------	-------------

Number of Functions	8 benchmark functions
Dimensionality	2D, 3D, 4D, 5D, 6D & 8D
Initial Training Data	20-50 points per function
Optimization Rounds	10 sequential rounds
Evaluations per Round	1 function evaluation per round
Total Evaluations	~30-90 evaluations per function (initial 20-50 + 10 rounds)
Total Dataset Size	Approximately 240-720 total data points across all 8 functions

2.2 Data Format and Structure

File Format:

X (Features):

- Shape: (n_samples, n_dimensions)
- Dimensions: 2 to 8
- Data Type: float64 (NumPy arrays or CSV)
- Format: Each row is one evaluation point

y (Objective Values):

- Shape: (n_samples,)
- Data Type: float64
- Format: Scalar value per evaluation
- Range: Function-dependent (typically [-100, 100] or similar)

Example Data Structure:

Initial Data (round 0):

```
X shape: (25, 8) # 25 initial points in 8D space
y shape: (25,)   # 25 corresponding function values
```

After Round 1:

```
X shape: (26, 8) # 25 initial + 1 new point
y shape: (26,)
```

After Round 10:

```
X shape: (35, 8) # 25 initial + 10 new points
y shape: (35,)
```

2.3 Benchmark Functions Included

The dataset contains 8 Black Box Optimization benchmark functions:

ID	Function Type	Dimension(s)	Optimization Type	Typical Range
F1	Benchmark Function 1	2D	Minimization	[0, 1]
F2	Benchmark Function 2	2D	Minimization	[0, 1]

F3	Benchmark Function 3	3D	Minimization	[0, 1]
F4	Benchmark Function 4	4D	Minimization	[0, 1]
F5	Benchmark Function 5	5D	Minimization	[0, 1]
F6	Benchmark Function 6	6D	Minimization	[0, 1]
F7	Benchmark Function 7	6D	Minimization	[0, 1]
F8	Benchmark Function 8	8D	Minimization	[0, 1]

2.4 Data Size and Specifications

Aggregate Statistics:

- **Total Training Points:** ~240-720 points (across all functions and rounds)
- **Per-Function Points:** ~30-90 points each
- **Dimensionality Range:** 2 to 8 dimensions
- **Total Features:** 8 (maximum dimensionality)
- **File Size Estimate:** <5 MB (for all functions combined in CSV/NPY format)

2.5 Known Gaps and Limitations

Data Representation:

- **Dimensionality Imbalance:** More 8D functions than lower dimensions (reflects challenge of high-dimensional optimization)
- **Sparse High-Dimensional Sampling:** Initial 20-50 points in 8D space provides limited coverage due to curse of dimensionality
- **No Noise:** All function evaluations are deterministic (no measurement noise or stochasticity)

Function Properties:

- **Continuity:** All functions are continuous (no discrete variables)
- **Differentiability:** Functions are smooth (no corners or discontinuities)
- **Bounds:** All functions have defined input bounds; no unbounded optimization
- **Single-Objective:** Only single objective per function (no multi-objective optimization)
- **No Constraints:** Functions have no equality or inequality constraints

Coverage:

- **Initial Sampling:** Latin Hypercube Sampling may not uniformly cover 8D space due to curse of dimensionality
- **Optimization Trajectory:** Only 10 refinement steps per function (limited trajectory data)

- **Function Diversity:** All functions are synthetic benchmarks (limited real-world complexity)

3. COLLECTION PROCESS

3.1 Data Generation Strategy

Phase 1: Initial Training Data

Method: Latin Hypercube Sampling (LHS)

```
sampler = qmc.LatinHypercube(d=8)
X_initial = qmc.scale(
    sampler.random(n=N_initial),
    l_bounds=[lower bounds per dimension],
    u_bounds=[upper bounds per dimension]
)
```

Specifications:

- **Sampling Method:** Latin Hypercube Sampling ensures space-filling coverage
- **Number of Initial Points:** 20-50 points per function
- **Bounds:** Function-specific parameter ranges from literature
- **Purpose:** Create diverse, well-distributed initial training set

Phase 2: Iterative Bayesian Optimization

Round Structure (10 rounds total, 1 evaluation per round):

1. Fit Gaussian Process Surrogate:

- Kernel: ConstantKernel(1.0) × Matern(v=2.5, length_scale=1.0)
- Regularization: alpha=0.01 (noise tolerance for stability)
- Optimizer: 30 restarts for hyperparameter optimization
- Input/Output Scaling: StandardScaler applied to both X and y (critical for GP)

2. Generate Candidate Pool:

- Method: Latin Hypercube Sampling
- Size: 30,000 candidate points
- Purpose: Explore search space broadly

3. Predict on Candidates:

- Use fitted GP to predict mean (μ) and std (σ) on all candidates
- Ensure predictions are rescaled to original units

4. Compute Acquisition Function:

- Function: Upper Confidence Bound (UCB)
- Formula: $UCB = \mu + \kappa \times \sigma$
- Parameter: $\kappa = 2.0$ (exploration/exploitation trade-off)

5. Select Next Point:

- Select point with highest UCB score
- Ensure point respects function bounds

6. Evaluate and Update:

- Evaluate selected point on true objective function
- Add to training dataset
- Record: X_new, y_new, round number, UCB score

7. **Repeat:** Iterations 1-6 for 10 rounds total

3.2 Temporal Information

Collection Timeline:

- **Data Collection Period:** [Start Date] to [End Date - To Be Filled]
- **Total Duration:** [Estimated: Several hours]
- **Computational Cost:** Single-threaded, <1 second per evaluation
- **Total Computation Time:** ~30 minutes for all functions × 10 rounds

Time Breakdown:

Initial Sampling:	~1 minute (LHS sampling × 8 functions)
First GP Fitting:	~2 minutes (8 functions)
Candidate Generation:	~1 minute (30K candidates × 8)
Optimization Rounds:	~20 minutes (10 rounds × 8 functions)
Analysis & Logging:	~5 minutes

Total: ~30 minutes

3.3 Detailed Sampling Procedure

Pseudocode for Collection:

```
# For each of 8 benchmark functions:
for function_id in range(1, 9):

    # Phase 1: Initial sampling
    X_initial = LatinHypercube(n=25,
d=dimensions[function_id])
    y_initial = [evaluate_function(x) for x in X_initial]

    X_train = X_initial.copy()
    y_train = y_initial.copy()

    # Phase 2: Optimization loop
    for round_num in range(1, 11):

        # Scale data
        X_scaled = StandardScaler().fit_transform(X_train)
        y_scaled = StandardScaler().fit_transform(y_train)

        # Fit GP
        kernel = ConstantKernel(1.0) × Matern(nu=2.5)
        gp = GaussianProcessRegressor(kernel=kernel,
alpha=0.01, n_restarts=30)
        gp.fit(X_scaled, y_scaled)

        # Generate candidates
        X_candidates = LatinHypercube(n=30000,
d=dimensions[function_id])
        X_candidates_scaled =
```

```

StandardScaler().transform(X_candidates)

    # Predict
    y_pred, y_std = gp.predict(X_candidates_scaled,
return_std=True)

    # Compute UCB
    ucb = y_pred + 2.0 * y_std
    best_idx = argmax(ucb)
    x_next = X_candidates[best_idx]

    # Evaluate
    y_next = evaluate_function(x_next)

    # Update
    X_train = append(X_train, x_next)
    y_train = append(y_train, y_next)

    # Log
    log(round_num, x_next, y_next, ucb[best_idx])

```

3.4 Data Quality Assurance

Validation Checks Performed:

Bounds Validation:

- All sampled points X remain within function-specific bounds
- No out-of-bounds evaluations

Numerical Stability:

- StandardScaler ensures input range $\approx [-1, 1]$
- Prevents numerical issues in GP kernel computation

Overfitting Detection:

- Monitor training R^2 score
- Flag if $R^2 > 0.99$ (suggests overfitting)
- Alpha=0.01 regularization mitigates this

Uncertainty Validation:

- Check that uncertainty estimates (σ) are reasonable
- Warn if uncertainty $\gg$ function range
- Verify $\sigma > 0$ everywhere (well-posed GP)

Sanity Checks:

- Predicted y range should overlap with training y range
- UCB scores should be reasonable (mean $\pm$ a few std)
- Candidate pool adequately samples the space

4. PREPROCESSING AND USES

4.1 Data Transformations Applied

1. Scaling (StandardScaler):

```
from sklearn.preprocessing import StandardScaler
```

```
scaler_X = StandardScaler()  
scaler_y = StandardScaler()
```

```
X_scaled = scaler_X.fit_transform(X)  
y_scaled = scaler_y.fit_transform(y.reshape(-1, 1)).ravel()
```

- **Transformation:** $X' = (X - \text{mean}) / \text{std}$
- **Applied to:** Both input features (X) and output (y)
- **Purpose:**
 - Gaussian Process requires normalized inputs for kernel computation
 - Improves numerical stability
 - Standardizes length scale hyperparameters
- **Inverse Transform:** Predictions rescaled back to original units for interpretation

Before Scaling:

```
X_raw range:      [0.234, 95.678]  
y_raw range:      [-1234.5, 5678.9]
```

After Scaling:

```
X_scaled range: [-2.134, 2.987] (mean ≈ 0, std ≈ 1)  
y_scaled range: [-1.891, 2.456] (mean ≈ 0, std ≈ 1)
```

2. Inverse Transform for Output:

```
y_pred_original =  
scaler_y.inverse_transform(y_pred_scaled.reshape(-1,  
1)).ravel()
```

- Converts predictions back to original function value scale
- Ensures interpretability of results

3. No Other Preprocessing:

- ✗ No missing value imputation (dataset has no missing values)
- ✗ No outlier removal (all points valid benchmark evaluations)
- ✗ No feature engineering (use raw function inputs)
- ✗ No dimensionality reduction (preserve full problem structure)

4.2 Intended Uses

Recommended Applications:

1. Algorithm Benchmarking:

- Compare Bayesian Optimization vs. Random Search vs. Grid Search
- Evaluate different acquisition functions (UCB, Expected Improvement, Probability of Improvement)
- Benchmark surrogate models (GP, Random Forest, Neural Networks)

2. Educational Purposes:

- Teaching Black Box Optimization concepts
- Demonstrating Gaussian Process behavior
- Visualizing acquisition function landscapes

3. Research and Development:

- Investigating dimensionality effects on optimization
- Testing new acquisition functions
- Studying surrogate model quality vs. convergence

4. Hyperparameter Optimization:

- Tuning optimization algorithm hyperparameters
- Understanding exploration vs. exploitation trade-offs
- Analyzing sensitivity to acquisition function parameters (κ)

5. Academic and Thesis Work:

- Foundation for capstone or thesis projects
- Reproducible baselines for comparison
- Well-documented methodology

4.3 Inappropriate or Unrecommended Uses

NOT Recommended For:

1. Direct Real-World Application:

- These are synthetic benchmarks; real problems have different structure
- No domain constraints, multi-objectives, or noise handling
- Results unlikely to transfer directly to production systems

2. Function-Specific Optimization:

- Optimal strategies learned here may not generalize
- Real-world functions often have exploitable structure
- Hand-tuned methods outperform these generic approaches

3. Claims Beyond Scope:

- Cannot claim this proves a method is "best" globally
- Limited to these 8 functions in these dimensionality ranges
- No statistical significance testing across larger function sets

4. Production Deployment:

- Not validated for safety-critical or high-stakes applications
- Lacks robustness testing, error handling, edge cases
- Should undergo validation on real objectives before deployment

5. Misleading Presentations:

- Don't report only best-case results
- Must acknowledge limitations and synthetic nature
- Include confidence intervals and variability metrics

4.4 Ethical Considerations

Transparency & Reproducibility:

- Full algorithm details and hyperparameters documented
- Random seed (42) specified for deterministic results
- Code provided for independent verification
- Acknowledges synthetic nature and limitations upfront

Limitations & Honesty:

- Clear documentation of what functions are included
- Explicit statement that these are benchmarks, not real problems
- Known gaps and failure modes documented
- No overstated claims about real-world performance

Accessibility & Attribution:

- Dataset open for academic and research use
- Proper citation required in derivative work
- Original sources acknowledged
- No proprietary or restricted data

5. DISTRIBUTION AND MAINTENANCE

5.1 Data Availability and Access

Primary Location:

GitHub Repository: <https://github.com/kalpeshpjadhav/Imperial-Capstone-Project--BBO>

Path: /data/BBO_dataset/

Format & Files:

- **Datasheet:** BBO_Datasheet.md (this file)
- **Model Card:** BBO_ModelCard.md (optimization approach documentation)
- **Jupyter Notebook:** BBO_Optimization.ipynb (full code and results)
- **Data Files:** bbo_functions_data.csv or .npz files
- **README:** README.md (quick start guide)

Access Method:

- Public repository (no authentication required)
- Clone: git clone [Your GitHub Link]
- Download: Available as zip archive

5.2 Terms of Use and Licensing

License: [To Be Selected: MIT / CC-BY-4.0 / GPL-3.0 / Other]

Usage Rights:

- Academic research and education
- Non-commercial projects
- Commercial projects (if license permits)
- Modification and redistribution (with attribution)

Requirements:

- **Citation:** Must cite this datasheet and authors in publications
- **Attribution:** Credit original authors and benchmark sources
- **Acknowledgment:** Acknowledge limitations and synthetic nature
- **Share-Alike:** If using CC-BY-SA, derivative work must use same license

5.3 Updates and Maintenance

Maintainer Information:

- **Primary Maintainer:** [Your Name]
- **Contact:** [Your Email - To Be Filled]
- **Affiliation:** [Your Institution/Organization]

Maintenance Plan:

- **Update Frequency:** [Not planned / As needed / Quarterly updates]
- **Version Control:** Tracked via Git; all changes logged

- **Bug Fixes:** Corrections made promptly and documented
- **Enhancement Requests:** Considered for future versions

Contribution Guidelines:

- Issues and bug reports: Submit via GitHub Issues
- Feature requests: Create GitHub Discussions
- Code contributions: Submit Pull Requests
- Feedback: Contact maintainer directly

Known Issues:

- [To be filled as issues are discovered]

5.4 Related Resources and References

Related Materials:

- **Model Card:** See BBO_ModelCard.md for optimization approach details
- **Code Repository:** Full implementation in BBO_Optimization.ipynb
- **Academic Papers:** Listed in references section below

Derivative Works:

- Citation of this dataset in subsequent work is requested
- Maintainer would appreciate being notified of citations

6. DOCUMENT METADATA

Attribute	Value
Dataset Name	BBO Benchmark Functions with Bayesian Optimization
Version	1.0
Release Date	April 2026
Last Updated	April 2026
Authors	Kalpesh Jadhav
Contact Email	kalpeshjadhav.jde@gmail.com
GitHub Repository	https://github.com/kalpeshpjadhav/Imperial-Capstone-Project--BBO

7. REFERENCES

Foundational Works on Datasheet Framework

1. Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). "Datasheets for Datasets." *Communications of the ACM*, 64(12), 86-92.

Bayesian Optimization & Gaussian Processes

1. Mockus, J. (1975). "On Bayesian Methods for Seeking the Extremum." *Optimization Techniques IFIP Technical Conference*, 400-404.
2. Jones, D. R., Schonlau, M., & Welch, W. J. (1998). "Efficient Global Optimization of Expensive Black-box Functions." *Journal of Global Optimization*, 13(2), 455-492.
3. Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & de Freitas, N. (2016). "Taking the Human Out of the Loop: A Review of Bayesian

Optimization." *Proceedings of the IEEE*, 104(2), 148-175.

4. Rasmussen, C. E., & Williams, C. K. (2006). *Gaussian Processes for Machine Learning*. MIT Press.

Benchmark Functions

1. [Cite the specific sources of your 8 benchmark functions - To Be Filled]

8. APPENDIX: EXAMPLE DATA POINTS

Sample Initial Data (First 5 points of Function 1 - 2D):

X1	X2	y	Round
0.31940389	0.76295937	1.32267704E-79	0 (initial)
0.5743292	0.8798981	1.03307824E-46	0 (initial)
0.7310236	0.73299988	7.71087511E-16	0 (initial)
0.8403534	0.26473161	3.34177101E-124	0 (initial)
0.6501140	0.68152635	-3.60606264E-03	0 (initial)
0.4104371	0.1475543	-2.15924904E-54	0 (initial)
0.3126911	0.07872278	-2.08909327E-91	0 (initial)
0.6834181	0.86105746	2.53500115E-40	0 (initial)
0.0825072	0.40348751	3.60677119E-81	0 (initial)
0.8838898	0.58225397	6.22985647E-48	0 (initial)
0.650114	0.681526	-3.60622E-03	1 (optimized)
0.07344	0.995966	1.27E-308	2 (optimized)
0.834771	0.764338	-2.18E-43	3 (optimized)
0.705169	0.346253	-2.53E-60	4 (optimized)
0.569943	0.115525	3.91328897E-81	5 (optimized)
0.388117	0.661867	-8.02884257E-42	6 (optimized)
0.777293	0.530146	-1.04490041E-22	7 (optimized)
0.461856	0.8277	-2.82001434E-48	8 (optimized)

0.507675	0.360105	1.22051909E-08	9 (optimized)
----------	----------	----------------	---------------

End of Datasheet for BBO Capstone Project

For questions or feedback, please contact: kalpeshjadhav.jde@gmail.com

Last Generated: April 19, 2026